

First NoSQL Database

A practical introduction to NoSQL databases and Amazon DynamoDB

Table of Contents

00 NoSQL Recap

Flexible data models, relational differences and common NoSQL structures.

01 Amazon DynamoDB

Tables, items, attributes, keys, scaling, protection and querying.

02 Practice Lab

Create a DynamoDB table, add flexible items and query the data.

Background

We would like to add more features to our streaming service. This requires a highly scalable database with very fast performance so that we can organise and collect large amounts of metadata about customer viewing habits.

Solution

One feature idea is a digital bookmark that lets customers continue watching shows where they left off. The data does not need to follow one rigid schema, making Amazon DynamoDB, a NoSQL database service, a good fit.

```
{  
  "userId": "8472",  
  "showId": "breaking-bad",  
  "episode": 4,  
  "position": 1902  
}
```

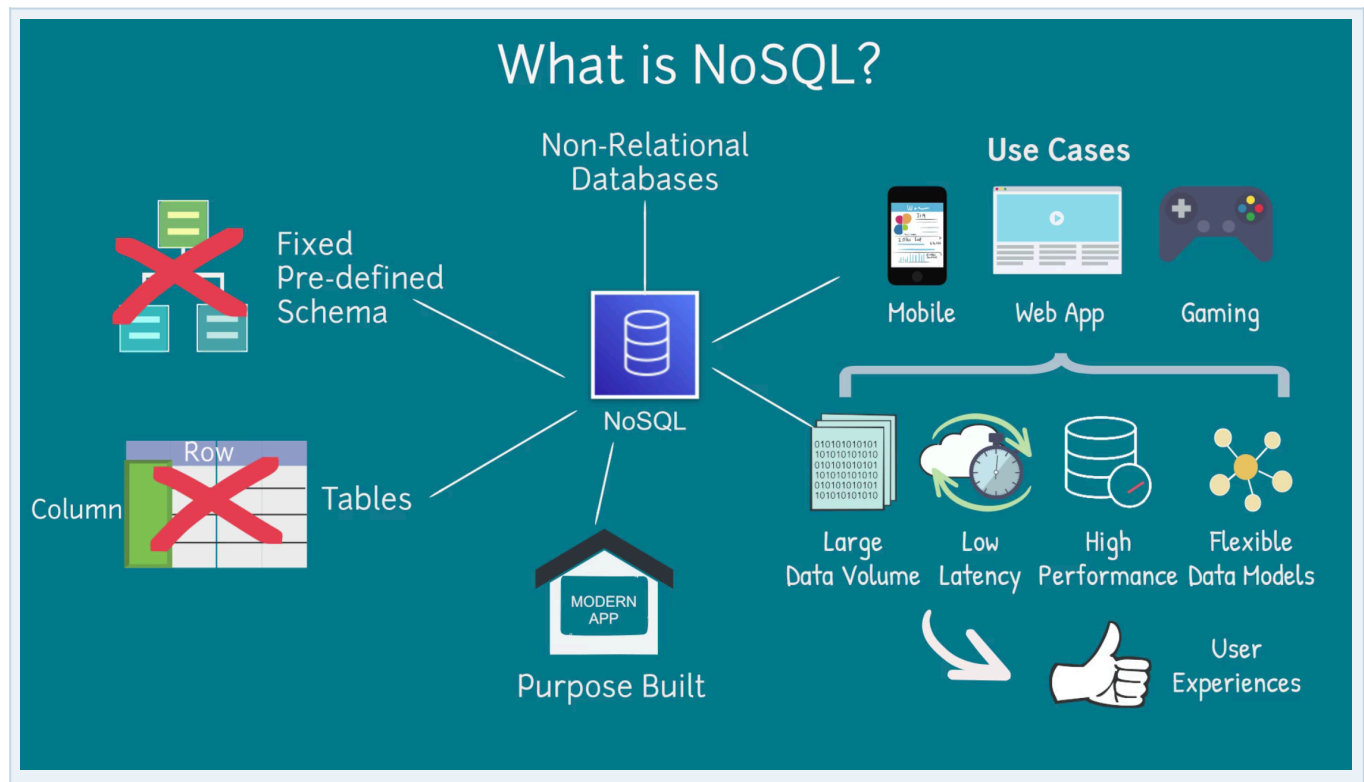
With millions of users constantly updating and retrieving bookmarks, we need huge numbers of simple reads and writes with very low latency. DynamoDB is designed for fast, predictable performance at very large scale and can store records with unique IDs and varying attributes.

00 NoSQL Recap

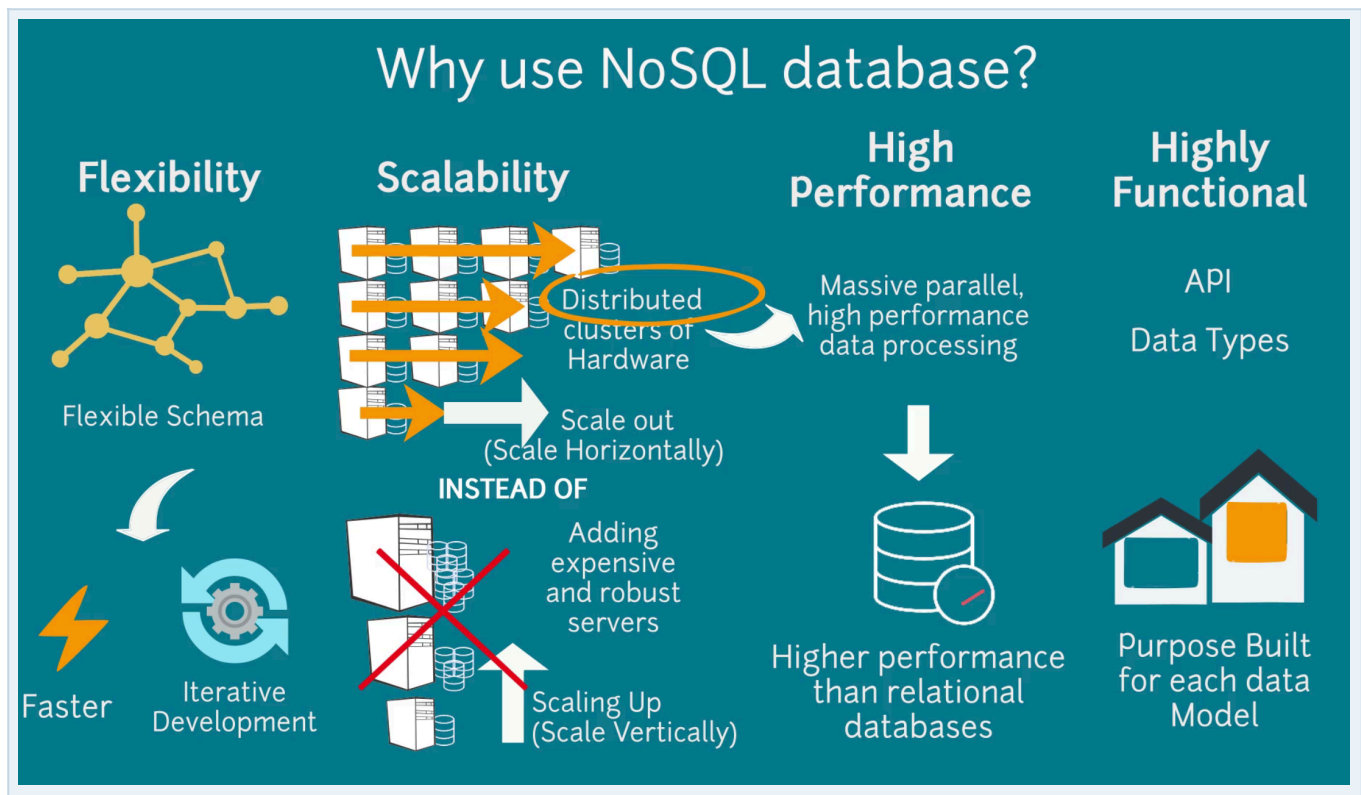
NoSQL databases provide a more flexible way of storing data than traditional relational databases. Instead of requiring every record to follow exactly the same structure, individual items can contain different attributes.

Item 1	Item 2
userId: 123	userId: 456
videoId: video1	videoId: video2
language: en-US	
device: Fire TV	device: Fire TV

Item 2 simply does not have a language attribute. In a relational database, the language column would still exist for the whole table even if its value were NULL.



This makes NoSQL flexible in structure, highly scalable, fast and easy to evolve. The trade-off is that complex relationships and flexible relational queries can become harder; SQL is often stronger for joins, reporting and complex relationships.



Relational vs NoSQL Databases

The main difference is how data is organised. A relational database separates related information into structured tables, while a NoSQL database can often keep related information together in a single item or document.

Relational example

ISBN (PK)	Title
1234	Dune

Here ISBN is the primary key used to identify the book.

Author ID	Name
77	Frank Herbert

The book and author are stored separately, so a relationship is needed to connect them:

Author ID	ISBN
77	1234

This relationship allows the database to understand that Frank Herbert wrote Dune. This is one reason they are called relational databases: information can be separated into tables and connected using keys.

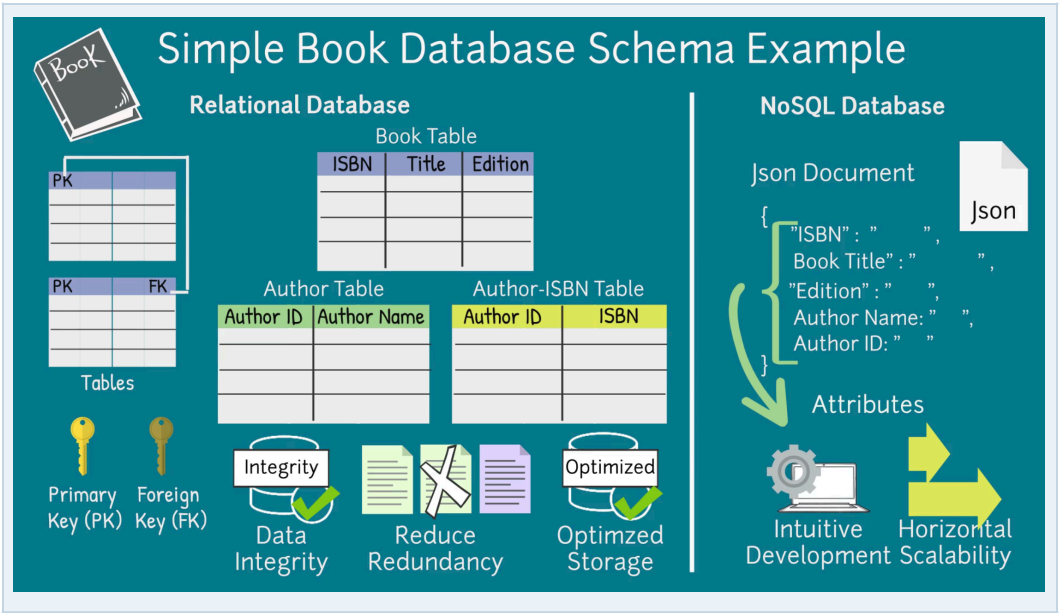
NoSQL example

Book Item	Value
ISBN	1234
Title	Dune
Author ID	77
Author Name	Frank Herbert

The application can retrieve the book and immediately get the author information without combining separate tables.

The trade-off

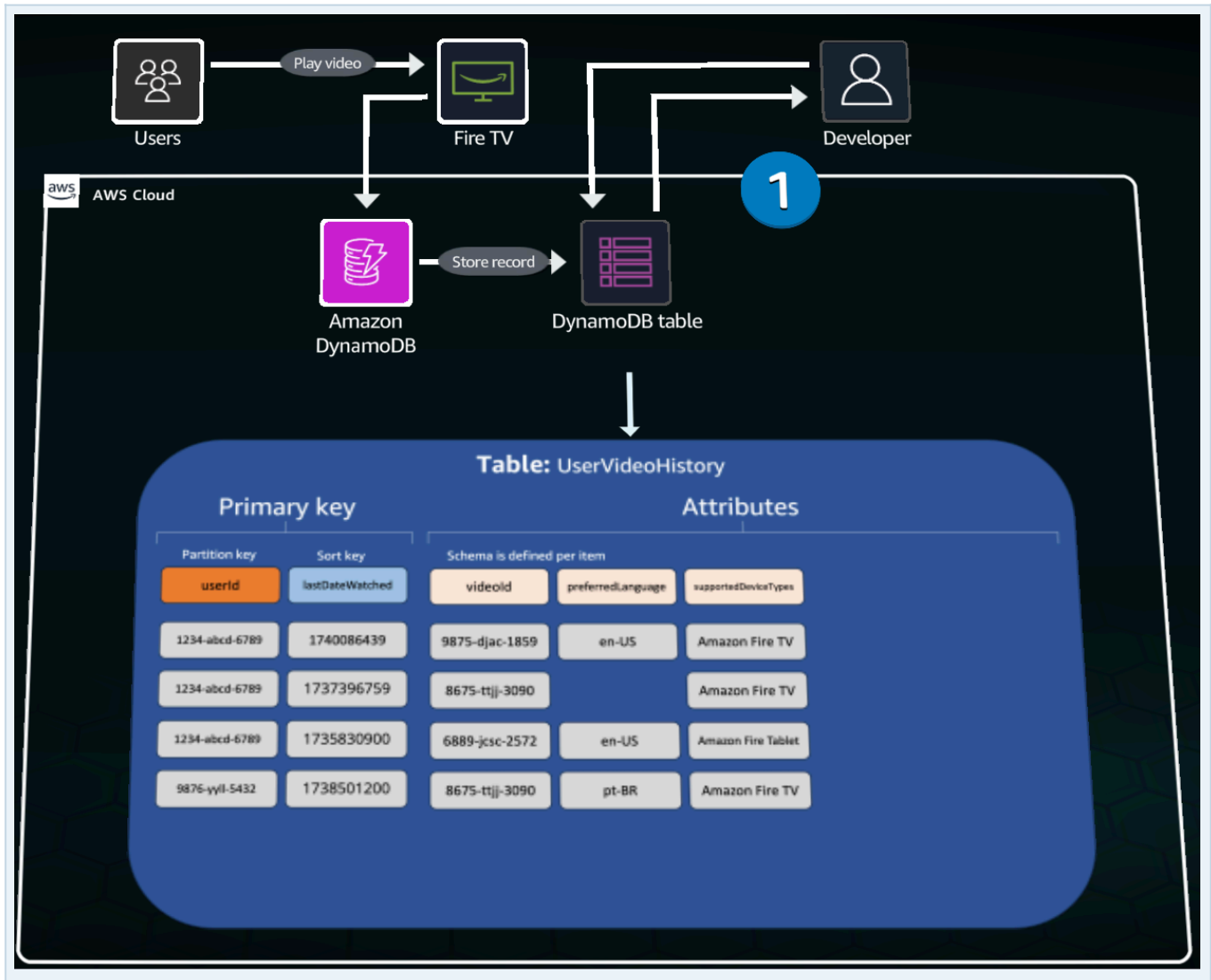
Relational	NoSQL
Data is often separated into related tables	Related data can often be stored together
Uses keys to connect records	Often avoids joins by storing needed data together
Reduces duplicated data	May intentionally duplicate data
Strong for complex relationships	Strong for simple, known access patterns
More rigid structure	More flexible structure



01 Amazon DynamoDB

DynamoDB Table

DynamoDB stores data in tables made up of individual records called items. Each item is identified using a primary key and can contain its own set of attributes.



In this example, the UserVideoHistory table stores the user, video watched, last watch time, preferred language and device type. Unlike a relational database, every item does not need to contain exactly the same attributes.

Types of NoSQL Databases

NoSQL is not one specific database structure. Different NoSQL models are designed for different workloads.

Model	How it stores data	Good for	AWS example
Key-Value	Key and value	Fast lookups, user data, gaming, IoT	DynamoDB
Document	JSON-like documents	Profiles, catalogs, content	DocumentDB
Graph	Connected nodes and relationships	Social networks, recommendations, fraud	Neptune
In-Memory	Data primarily in RAM	Caching, sessions, real-time data	ElastiCache / DAX
Search	Indexed data	Text search, logs, analytics	OpenSearch

Key-Value

Everything is retrieved using a unique key.

```
user-123 { video: "Dune", position: 1832 }
user-456 { video: "Alien", position: 940 }
```

Ask for user-123 and the database can quickly retrieve its value. Example: DynamoDB.

Document

Stores related information together as a document, commonly JSON-like:

```
{ "userId": 123, "name": "Alice", "language": "English",
  "favorites": ["Dune", "Alien"]
}
```

Different documents can contain different fields. Example: Amazon DocumentDB.

Graph

Designed around relationships between things.

```
Alice - FRIENDS_WITH - Bob
Alice - LIKES - Dune
Bob - LIKES - Alien
```

This makes questions such as “What movies do Alice’s friends like?” natural to model. Example: Amazon Neptune.

In-Memory

Keeps frequently needed data in RAM so it can be accessed extremely quickly.

```
session-123: shopping cart [popcorn, Dune Blu-ray]
```

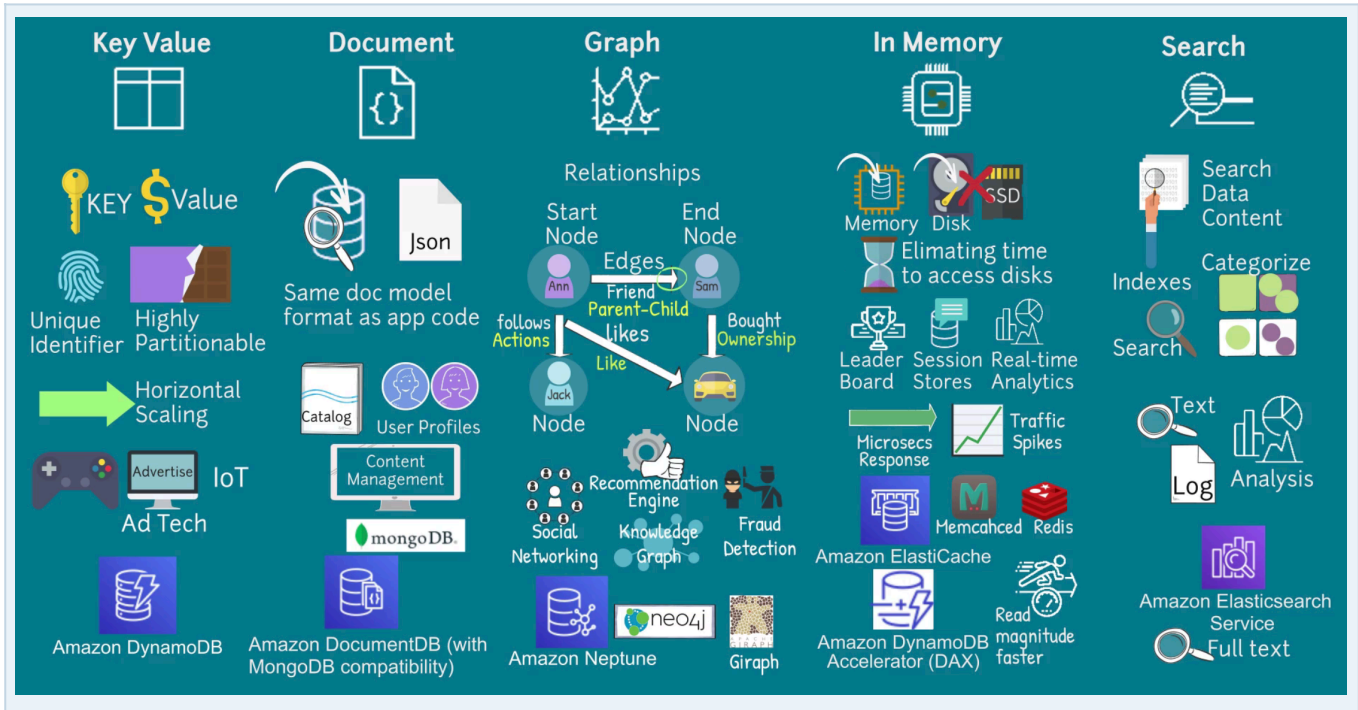
Useful for temporary information such as sessions and caching. Example: Amazon ElastiCache or DynamoDB Accelerator (DAX).

Search

Designed to search through the contents of large amounts of data rather than retrieve only by an exact key.

Search: "Dune" -> Dune, Dune Messiah, Children of Dune

Useful for text search, filtering, logs and analytics. Example: Amazon OpenSearch Service.

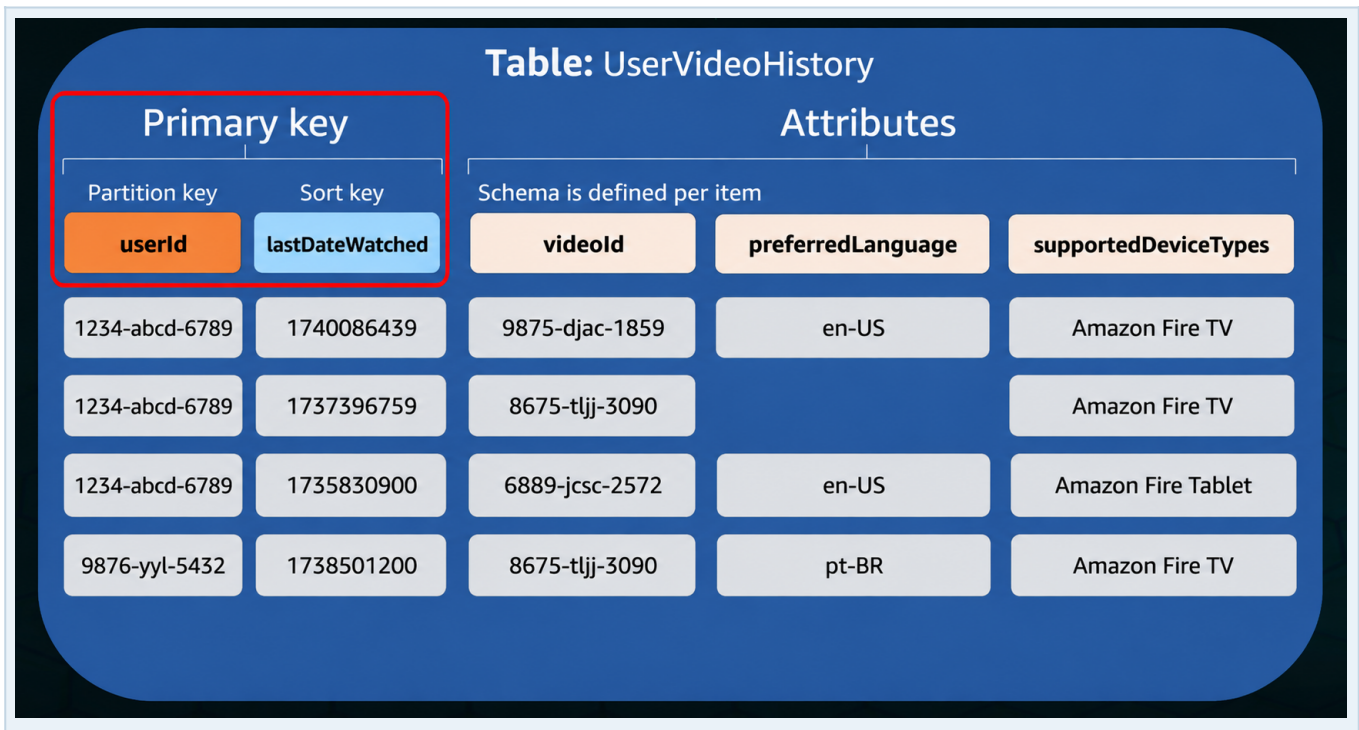


Composite Primary Keys

For our DynamoDB watch-history table, we will use a composite primary key combining:

- Partition key: `userId`
- Sort key: `lastDateWatched`

DynamoDB supports both a simple primary key using only a partition key and a composite primary key using a partition key plus a sort key.



Why do we need two keys?

Imagine we want to ask DynamoDB: “Give me all the videos watched by user-123, ordered by when they were watched.” We could store the entire history as one growing item, but then every update modifies that same large object.

```
user-123 { Dune: 10:30, Alien: 12:45, Shrek: 15:20 }
```

Instead, DynamoDB can store each watch event as its own item:

userId	lastDateWatched	videoId
user-123	10:30	Dune
user-123	12:45	Alien
user-123	15:20	Shrek

But I thought NoSQL meant documents?

NoSQL is a broad category, not one specific storage model. MongoDB is primarily document-oriented, while DynamoDB supports key-value and document-style data but is commonly designed around separate items identified by keys.

```
{ "userId": 123, "watchHistory":  
  [{"time":"10:30","video":"Dune"}, {"time":"12:45","video":"Alien"}]  
}
```

In our DynamoDB example, each watch-history entry is a separate item. The items are logically grouped by sharing the same partition key. The sort key distinguishes and orders the items within that group.

Unique identifier

With a composite primary key, partition key + sort key together form the unique identifier for an item.

DynamoDB Table Structure

A DynamoDB table is a collection of items. Each item represents one piece of data, and the information inside an item is stored as attributes.

Term	Meaning
Table	The entire collection of items
Item	One complete record
Attribute	An individual piece of information, such as videoId
Partition key	Mandatory. Used to group and locate data
Sort key	Optional. Distinguishes multiple related items sharing a partition key

Partition Keys and Sort Keys

With millions of watch-history records, DynamoDB needs a way to quickly find one user's records and a way to distinguish between that user's individual records.

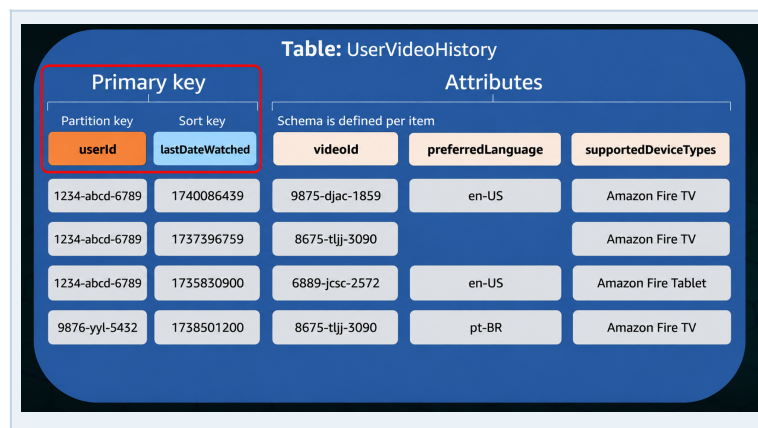
A simple partition key can be enough when one key identifies one complete item. When several items share the same partition key, the sort key becomes part of the primary key.

Partition Key	Sort Key	Result
userId: 123	video: Dune	Unique item
userId: 123	video: Alien	Unique item
userId: 123	video: Shrek	Unique item
userId: 456	video: Dune	Unique item
userId: 123	video: Dune	Already exists - overwrites that key

Using the lab's actual key design:

Partition Key <code>userId</code>	Sort Key <code>lastDateWatched</code>	Unique Primary Key
1234-abcd-6789	1740086439	1234-abcd-6789 + 1740086439
1234-abcd-6789	1737396759	1234-abcd-6789 + 1737396759
1234-abcd-6789	1735830900	1234-abcd-6789 + 1735830900
9876-yvll-5432	1738501200	9876-yvll-5432 + 1738501200

We now have multiple unique primary keys for watch-history items belonging to the same user.

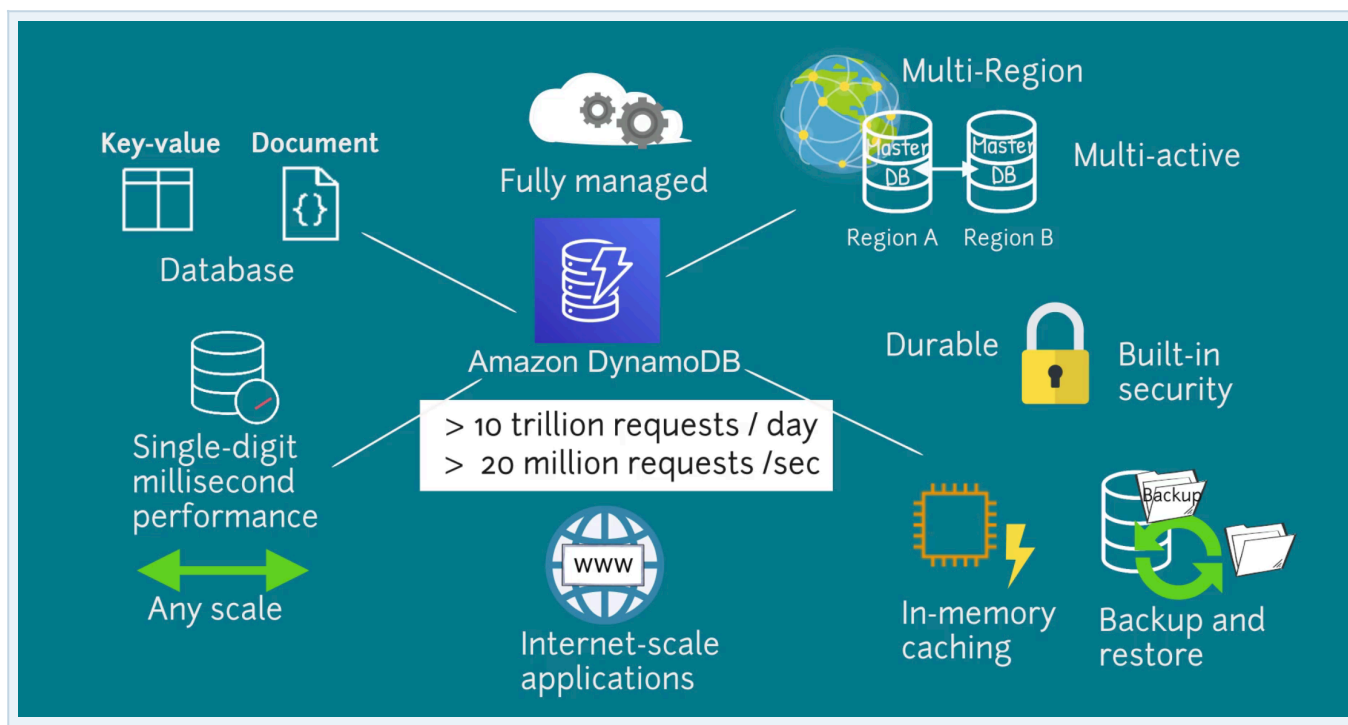


DynamoDB

Amazon DynamoDB is a fully managed NoSQL database built for fast performance and massive scale.

Feature	Meaning
Key-value & document	Supports flexible NoSQL data
Fully managed	AWS manages the infrastructure
Fast	Single-digit millisecond performance
Scalable	Handles very large workloads
Multi-Region	Can replicate data across Regions
Secure & durable	Built-in security and reliability
Caching	Supports in-memory caching
Backup & restore	Built-in data recovery

DynamoDB is ideal when you need fast, scalable data access without managing database infrastructure.

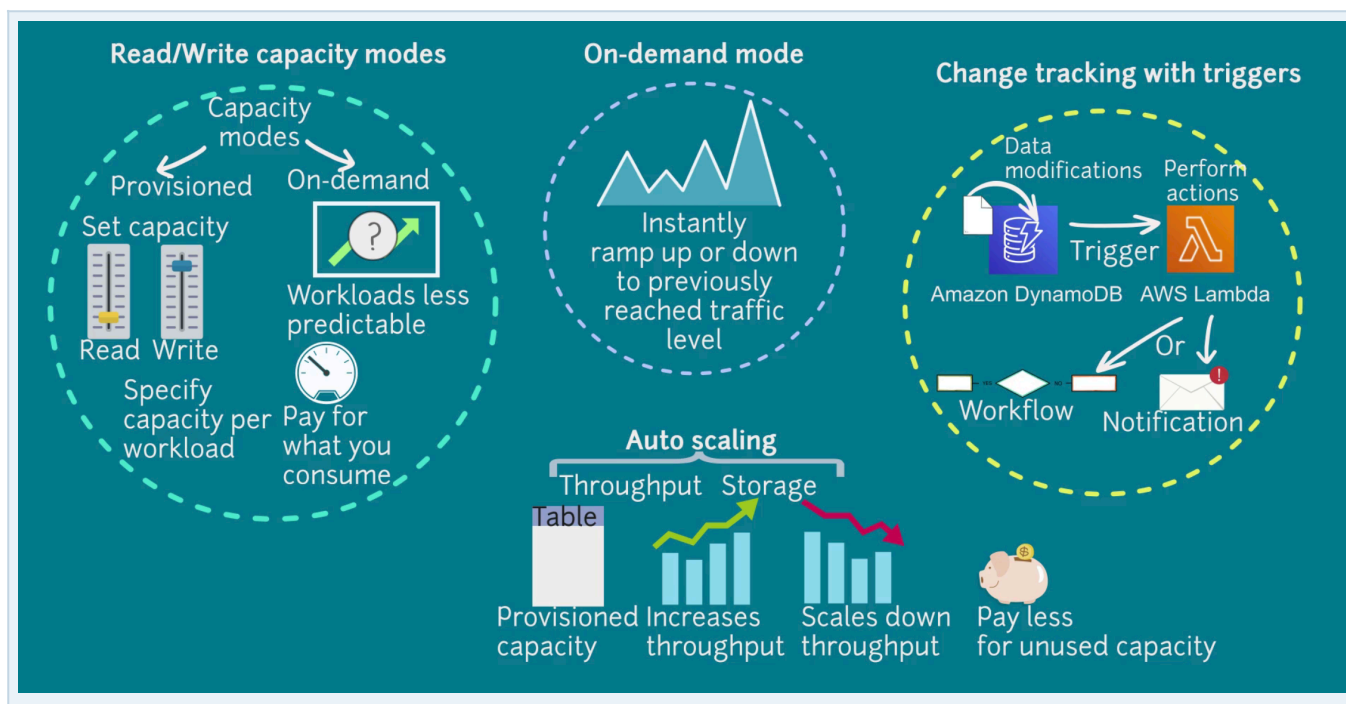


DynamoDB can scale with application traffic and can react automatically when data changes.

Feature	What it means
Provisioned capacity	You specify how much read/write capacity you expect
On-demand capacity	DynamoDB adjusts capacity automatically; you pay for requests
Auto scaling	Provisioned capacity can increase or decrease with demand
Triggers	Data changes can trigger actions such as an AWS Lambda function

In short

Choose provisioned capacity for more predictable workloads and on-demand capacity when traffic is less predictable. DynamoDB can also trigger other AWS services when data changes.

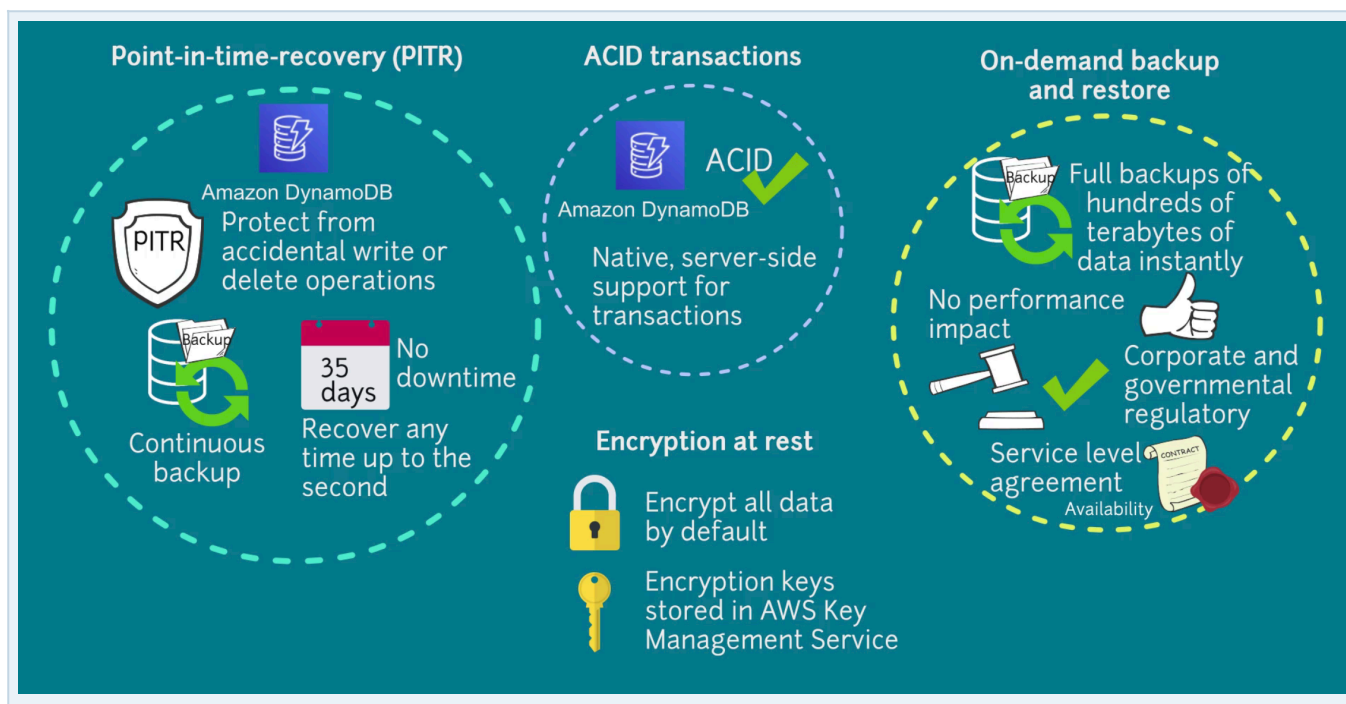


DynamoDB Data Protection

DynamoDB includes built-in features for recovering, protecting and safely modifying data.

Feature	What it means
Point-in-time recovery (PITR)	Continuous backups with recovery to a specific point in time
ACID transactions	Multiple operations can be performed safely as one transaction
Encryption at rest	Data is encrypted by default using AWS KMS
On-demand backups	Create full backups without affecting performance
High availability	Designed to provide reliable database availability

In short, DynamoDB provides backup, recovery, encryption and transaction support without requiring you to manage the underlying infrastructure.



Querying Data in DynamoDB

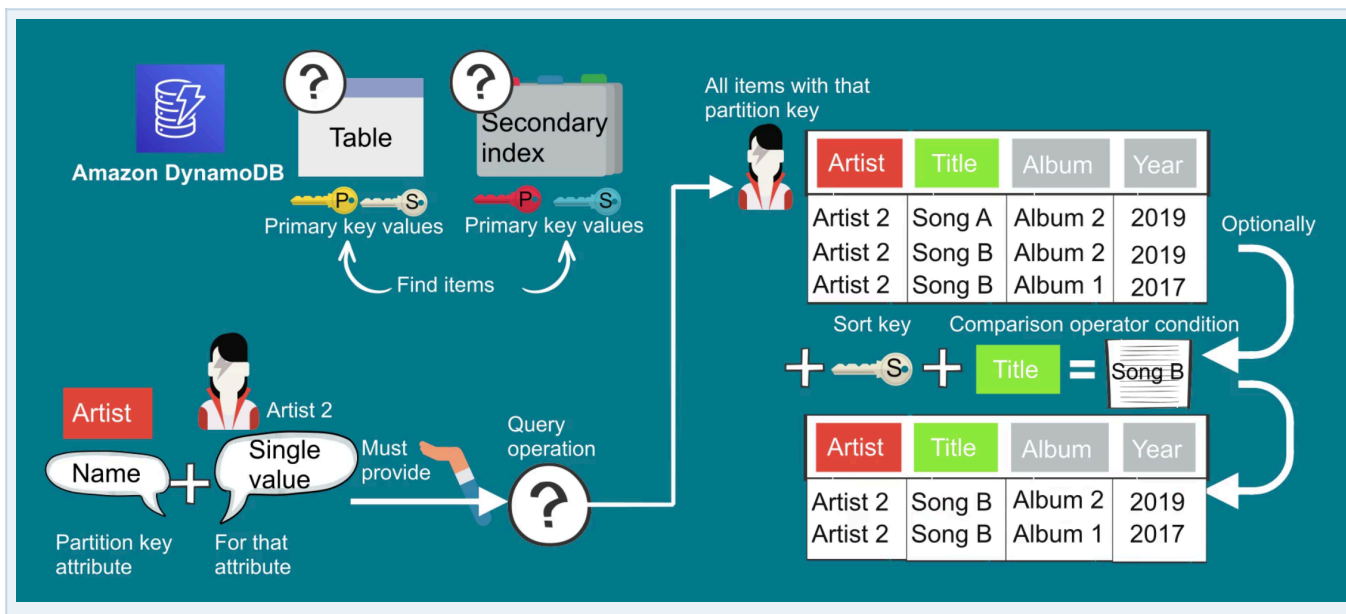
A Query finds items using their primary key. You must provide the partition key value, while the sort key can optionally narrow the results.

Part	What it does
Partition key	Required. Selects items with that value
Sort key	Optional. Narrows items within the selected partition
Secondary index	Lets you query using alternative key attributes

For example, Artist = "Artist 2" returns items for Artist 2. Adding Title = "Song B" narrows the result to that song.

Mental model

The partition key finds the group. The sort key optionally narrows which items inside that group you want.



The query

A DynamoDB query first uses the key condition to find items by their primary key. An optional filter expression can then remove unwanted items from those results.

```
aws dynamodb query \
  --table-name Music \
  --key-condition-expression "Artist = :a and SongTitle = :t" \
  --filter-expression "Year > :yyyy"
```

Part	Purpose
dynamodb query	Queries DynamoDB
--table-name Music	Selects the Music table
Artist = :a	Specifies the required partition key
SongTitle = :t	Narrows results using the sort key
Year > :yyyy	Optionally filters the returned results

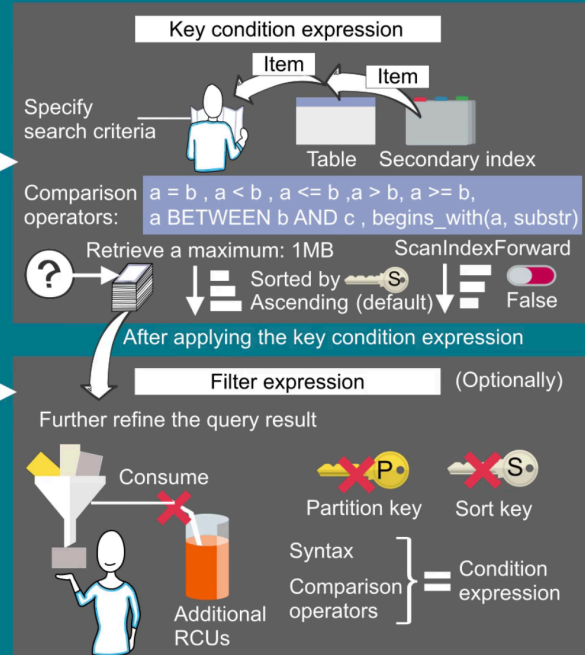
:a and :t are placeholders. They could be replaced with clearer placeholder names if preferred.

It is worth mentioning that querying can also be done using AWS built-in system.

Query Example

Using the AWS Command Line Interface (AWS CLI),
Query the "Music" table

```
aws dynamodb query \
  --table-name Music \
  --key-condition-expression "Artist = :a and SongTitle = :t" \
  --filter-expression "Year > :yyyy" \
```



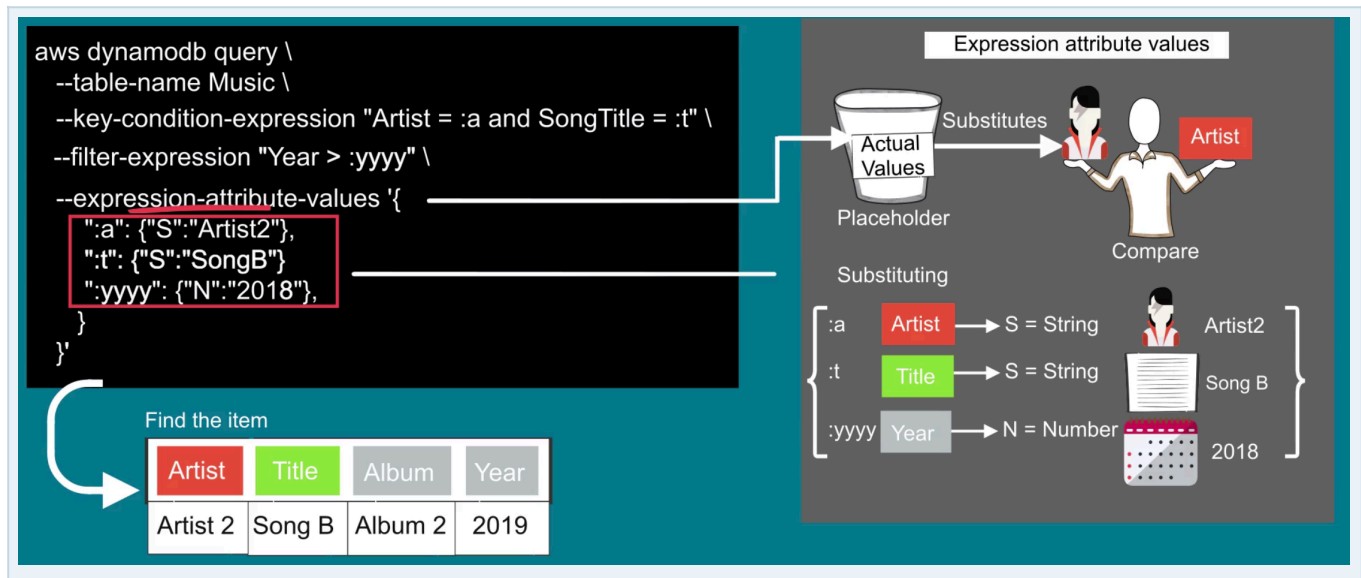
Expression Attribute Values

The placeholders receive their actual values through `--expression-attribute-values`.

```
--expression-attribute-values '{ ":a":{"S":"Artist2"}, ":t":{"S":"SongB"}, ":yyyy": {"N":"2018"} }'
```

Placeholder	Type	Actual value	Used for
:a	S = String	Artist2	Artist = :a
:t	S = String	SongB	SongTitle = :t
:yyyy	N = Number	2018	Year > :yyyy

The query effectively becomes Artist = "Artist2", SongTitle = "SongB", with a filter of Year > 2018.



Adding New Attributes

DynamoDB has a flexible schema, so new attributes can be added to individual items without changing the structure of the entire table.

```
Item 1 { userId: 123, videoId: "Dune", lastStopTime: 90 }
Item 2 { userId: 123, videoId: "Alien" }
```

Item 2 does not need a lastStopTime attribute at all. Non-key attributes can vary between items.

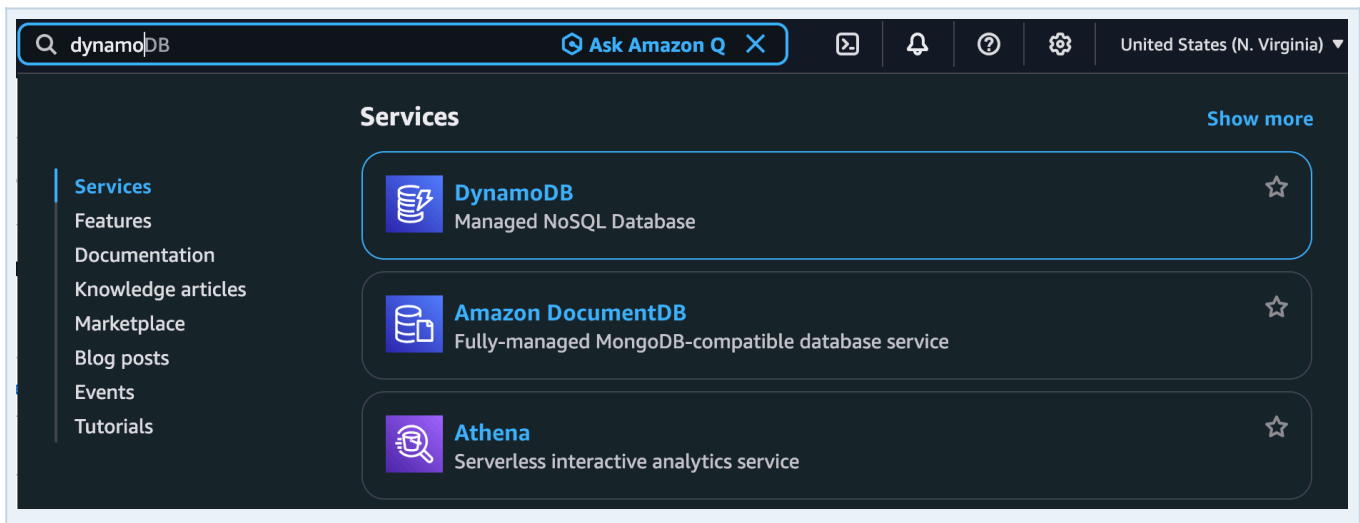
02 Practice Lab

In this practice lab we will:

- Create a NoSQL database as an Amazon DynamoDB table.
- Add records with a dynamic schema to the DynamoDB table.
- Query the DynamoDB table.

Create the DynamoDB Table

Navigate to DynamoDB using the AWS search field.



In the left navigation pane, choose Tables and create a new table.

Setting	Value
Table name	UserVideoHistory
Partition key	userId (String)
Sort key	lastDateWatched (Number)

Table details
[Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
 This will be used to identify your table.

UserVideoHistory

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
 The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

userId

String

1 to 255 characters and case sensitive.

Sort key - optional
 You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

lastDateWatched

Number

1 to 255 characters and case sensitive.

Keep the default settings, create the table, and wait until it is up and running.

Add the First Item

Navigate to your table.

Tables (1)
[Info](#)

Last updated
 September 8, 2026, 14:05 (UTC+2:00)

Actions
 Delete
 Create table

Find tables

Filter by tag
 Any tag key

Filter by tag value
 Any tag value

< 1 >

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite
☐	UserVideoHistory	Active	userId (S)	lastDateWatched (N)	0	0	Off	☆

Open Actions and create a new item.

Actions
 Explore table items

Create item
 Delete table
 Indexes
 Create global secondary index
 Create vector index
 Settings
 Edit capacity
 Turn on TTL
 Update table class
 Manage tags
 Global tables
 Create replica

Create the item using the provided test attributes. Since this is hardcoded lab data, we can add the attributes directly.

Create item

FormJSON view

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes

Add new attribute

Attribute name	Value	Type	Remove
userId - Partition key	12345-abcd-6789	String	
lastDateWatched - Sort key	1740086439	Number	

Cancel

Create item

DynamoDB lets us add additional non-key attributes to an item now or later.

Attributes

Add new attribute

Attribute name	Value	Type	Remove
userId - Partition key	12345-abcd-6789	String	
lastDateWatched - Sort key	1740086439	Number	
videoid	9875-djac-1859	String	Remove
preferredLanguage	en-US	String	Remove

Cancel

Save

Save and close

You can also add a List attribute, much like an array.

supportedDeviceTypes

Insert a field

List

Value 0	Amazon Fire TV	String
Value 1	Amazon Fire Tablet	String

Add lastStopTime

Review the table design again:

Table: UserVideoHistory				
Primary key		Attributes		
Partition key	Sort key	Schema is defined per item		
userId	lastDateWatched	videoid	preferredLanguage	supportedDeviceTypes
1234-abcd-6789	1740086439	9875-djac-1859	en-US	Amazon Fire TV
1234-abcd-6789	1737396759	8675-tljj-3090		Amazon Fire TV
1234-abcd-6789	1735830900	6889-jcsc-2572	en-US	Amazon Fire Tablet
9876-yy1-5432	1738501200	8675-tljj-3090	pt-BR	Amazon Fire TV

While we do all the Primary keys and attributes listed above..
We are still missing the requested lastStopTime attribute from the mission statement. Add it as a Number.

Attributes

Add new attribute

Attribute name	Value	Type	Remove
userId - Partition key	12345-abcd-6789	String	
lastDateWatched - Sort key	1740086439	Number	
preferredLanguage	en-US	String	Remove
supportedDeviceTypes	Insert a field	List	Remove
videoid	9875-djac-1859	String	Remove
lastStopTime	90	Number	Remove

Cancel

Save

Save and close

Query the Table

Save the changes and change the operation from Scan to Query.

UserVideoHistory

Autopreview

View table details

▼ Scan, query or search items

Info

Scan

Query

Search

Query the items to verify that the setup works.

Key	Condition / value
userId (partition key)	12345-abcd-6789
lastDateWatched (sort key)	Greater than 1740086438

Partition key

Attribute

Value

userId

12345-abcd-6789

Sort key - optional

Attribute

Value

lastDateWatched

Greater than or equal to

▼

1740086438

Sort descending

Results

✔ **Completed** · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 0,5

Table: UserVideoHistory - Items returned (1)

Query started on September 08, 2026, 14:21:09

Actions

Create item

< 1 >

⚙

☐	userId (String)	lastDateWatched (Number)	lastStopTime	preferredLanguage
☐	12345-abcd-6789	1740086439	90	en-US

When do we use Scan?

A Scan operation reads every item in a table or secondary index.

- Returns matching items and their attributes.
- Stops when scanned data reaches the 1 MB limit.
- Provides a LastEvaluatedKey so scanning can continue where it stopped.
- Can return no matching results if the filter excludes all scanned items.

Query vs Scan

Query is generally preferred when you know the partition key because DynamoDB can go directly to the relevant items. Scan examines the table much more broadly.

Create a New Item + Attribute

Earlier we established that two items in the same DynamoDB table can contain different non-key attributes. Let's test this by creating another item.

Reuse the same basic structure as before, but add a new rating attribute of type Number.

Attributes

Add new attribute ▼

Attribute name	Value	Type	Remove
userId - Partition key	12345-zyxw-6789	String	
lastDateWatched - Sort key	1740086322	Number	
lastStopTime	45	Number	Remove
preferredLanguage	en-SE	String	Remove
supportedDeviceTypes	Insert a field ▼	List	Remove
videoid	9875-djac-1859	String	Remove
rating	5	Number	Remove

Cancel>Create item

What this demonstrates

The new item can contain rating even if the earlier item does not. DynamoDB's flexible schema allows individual items in the same table to have different non-key attributes.

Summary

In this section, we explored why DynamoDB is useful for applications that need fast, scalable and flexible data storage, using a streaming service and its watch history as our main example.

We learned that NoSQL is not limited to document databases like MongoDB and that DynamoDB instead organises data into individual items containing flexible attributes.

We also learned how partition and sort keys work together to uniquely identify items, and how these keys can be used to efficiently query specific data. Because DynamoDB has a flexible schema, new attributes such as 'lastStopTime' can be added without changing every existing item.

The main takeaway is that DynamoDB is designed around how the application needs to access its data. Choosing appropriate keys and structuring items around those access patterns allows DynamoDB to provide fast and scalable data access without requiring us to manage the underlying database infrastructure.